Laporan Tugas Besar 2

IF2211 Strategi Algoritma

Pemanfaatan Algoritma BFS dan DFS dalam Mekanisme Penelusuran CSS pada Pohon Document Object Model

Semester II Tahun 2025/2026

Kelompok highAlcohol

Stevanus Agustaf Wongso
13524020

Jonathan Kris Wicaksono
13524023

Philipp Hamara
13524101

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1 Deskripsi Tugas **1**
1.1 Latar Belakang . . . 1
1.2 Spesifikasi Tugas . . . 1
1.2.1 Spesifikasi Wajib . . . 1
1.2.2 Spesifikasi Bonus . . . 1

2 Landasan Teori **3**
2.1 HTML dan Document Object Model . . . 3
2.1.1 Proses *HTML Parsing* . . . 3
2.2 CSS Selector . . . 3
2.2.1 Selector Dasar (*Simple Selector*) . . . 3
2.2.2 Kombinator . . . 3
2.3 Graf dan Pohon . . . 4
2.4 Breadth First Search (BFS) . . . 4
2.5 Depth First Search (DFS) . . . 4
2.6 Perbandingan BFS dan DFS . . . 5
2.7 *Lowest Common Ancestor* dengan *Binary Lifting* . . . 5

3 Aplikasi BFS dan DFS pada Penelusuran Pohon DOM **7**
3.1 Pemodelan Pohon DOM . . . 7
3.2 Pemetaan Persoalan ke Elemen Algoritma Traversal . . . 7
3.3 Pencocokan CSS Selector terhadap Pohon DOM . . . 7
3.3.1 Parsing Selector . . . 7
3.3.2 Algoritma Pencocokan (Right-to-Left) . . . 8
3.4 Strategi Traversal BFS/DFS dengan Satu *Frontier* . . . 8
3.5 Analisis Efisiensi dan Perbandingan BFS vs DFS . . . 9

4 Implementasi dan Pengujian **10**
4.1 Arsitektur Perangkat Lunak . . . 10
4.2 Implementasi Backend (Go) . . . 11
4.2.1 Organisasi Berkas . . . 11
4.2.2 Pseudocode Inti . . . 11
4.2.3 Struktur Data Utama Backend . . . 13
4.2.4 Fungsi dan Prosedur Utama . . . 14
4.3 Implementasi Frontend (Next.js + TypeScript) . . . 14
4.3.1 Animasi Penelusuran . . . 14
4.4 Pengujian . . . 15
4.4.1 Strategi Pengujian . . . 15
4.4.2 Hasil Pengujian Fungsional . . . 15
4.4.3 Pengujian Kinerja: BFS vs DFS . . . 15
4.4.4 Analisis Hasil Pengujian . . . 15

5 Kesimpulan dan Saran **17**
5.1 Kesimpulan . . . 17
5.2 Saran . . . 17

Lampiran **18**

Daftar Pustaka **20**

1 Deskripsi Tugas

1.1 Latar Belakang

Document Object Model (DOM) adalah representasi terstruktur dari sebuah dokumen HTML dalam bentuk pohon yang memungkinkan setiap elemen pada halaman web untuk diakses, dimodifikasi, dan dimanipulasi secara terprogram. Struktur ini merepresentasikan relasi antar elemen HTML (*parent*, *child*, dan *sibling*) sehingga sangat memudahkan pengolahan dokumen secara otomatis. Elemen `<html>` berperan sebagai *root* yang membawahi dua bagian utama, yaitu `<head>` dan `<body>`.

Visualisasi dari pohon DOM dideklarasikan melalui *Cascading Style Sheets* (CSS). Untuk mentargetkan elemen yang akan diberi gaya, CSS menggunakan mekanisme *CSS Selector*. Mekanisme pencocokan selector terhadap pohon DOM pada dasarnya adalah proses *graph traversal* terhadap struktur pohon.

Pada Tugas Besar 2 IF2211 Strategi Algoritma ini, mahasiswa diminta membangun aplikasi berbasis web yang melakukan penelusuran CSS pada pohon DOM menggunakan dua algoritma klasik, yaitu *Breadth First Search* (BFS) dan *Depth First Search* (DFS). Aplikasi menerima input berupa URL atau teks HTML mentah, *selector*, algoritma, dan jumlah hasil yang kemudian menghasilkan visualisasi pohon DOM, jalur traversal yang di-*highlight*, metrik waktu dan banyaknya node yang dikunjungi, serta *traversal log*.

1.2 Spesifikasi Tugas

1.2.1 Spesifikasi Wajib

Aplikasi yang dibangun harus memenuhi ketentuan-ketentuan berikut:

1. Aplikasi berbasis web dengan *frontend* bebas dan *backend* dalam salah satu bahasa yang diperbolehkan (Go, C#, Rust, atau TypeScript).
2. Mampu melakukan *scraping* HTML dari URL yang diberikan pengguna, atau menerima input teks HTML mentah secara langsung.
3. Input aplikasi meliputi:
   - URL atau teks HTML sendiri
   - Pilihan algoritma traversal (BFS atau DFS)
   - CSS selector
   - Jumlah hasil (*top n* atau seluruh kemunculan)
4. HTML diparsing menjadi struktur pohon DOM.
5. Output yang diharapkan:
   - Visualisasi struktur pohon DOM beserta kedalaman maksimumnya.
   - *Highlight* pada jalur traversal yang ditempuh algoritma.
   - Waktu pencarian dan jumlah node yang dikunjungi.
   - *Traversal log* yang menyimpan tahapan pencarian.
6. Program disertai README, dan dapat dikompilasi pada Windows maupun Linux.

1.2.2 Spesifikasi Bonus

Bonus yang tersedia (maks. 20 poin) terdiri dari:

1. **Video (4 poin):** Video demo aplikasi dengan audio dan wajah seluruh anggota kelompok, diunggah ke YouTube.
2. **Deploy (5 poin):** *Deployment* aplikasi ke Microsoft Azure Virtual Machine sehingga dapat diakses secara publik.
3. **Docker (2 poin):** Seluruh aplikasi (*frontend* dan *backend*) dijalankan melalui kontainer Docker.
4. **Animasi (6 poin):** Menampilkan animasi penelusuran pohon DOM untuk setiap jenis pencarian.

5. **Multithreading (3 poin):** Implementasi paralelisasi pada BFS/DFS untuk memproses beberapa node secara bersamaan.

6. **LCA Binary Lifting (3 poin):** Implementasi algoritma *Lowest Common Ancestor* dengan teknik *binary lifting* untuk menentukan leluhur bersama terendah dari dua elemen DOM.

2 Landasan Teori

2.1 HTML dan Document Object Model

HyperText Markup Language (HTML) adalah bahasa markup yang digunakan untuk mendeskripsikan struktur dokumen web. Setiap elemen HTML ditulis dengan pasangan *tag* pembuka dan penutup, misalnya `<p>...</p>`. Beberapa tag bersifat *self-closing* (*void element*) seperti `<br>`, `<img>`, dan `<input>`. Tag pembuka dapat memuat atribut (`id`, `class`, `href`, dst.) yang memberikan informasi tambahan mengenai elemen.

Pada saat *browser* memuat dokumen HTML, dokumen tersebut diproses oleh *HTML parser* yang mengubah untaian karakter menjadi struktur data pohon *Document Object Model* (DOM). Dalam DOM, setiap tag HTML direpresentasikan sebagai sebuah *node*. Relasi antar tag yang bersarang direpresentasikan sebagai hubungan *parent-child*, dan tag yang berada pada tingkat yang sama di bawah parent yang sama menjadi *sibling*. Node teratas adalah *root*, umumnya berupa elemen `<html>`.

2.1.1 Proses *HTML Parsing*

Secara umum proses *parsing* terdiri atas dua tahap utama:

1. **Tokenisasi (*lexing*):** dokumen HTML dibaca secara karakter-per-karakter dan dikelompokkan menjadi token-token seperti *open tag*, *close tag*, *self-closing tag*, komentar, `DOCTYPE`, dan *text node*.

2. **Pembentukan pohon:** token dieksekusi secara berurutan menggunakan *stack*. Setiap *open tag* mendorong node baru ke *stack*, *close tag* mem-*pop* satu level, dan *self-closing tag* dilampirkan ke *top-of-stack* tanpa mem-*push*.

Struktur pohon yang dihasilkan inilah yang disebut pohon DOM dan menjadi objek yang di-*traverse* oleh algoritma BFS/DFS.

2.2 CSS Selector

Cascading Style Sheets (CSS) adalah bahasa stylesheet yang digunakan untuk mengatur tampilan elemen HTML. Pemilihan elemen yang akan di-*style* dilakukan melalui mekanisme CSS *selector*.

2.2.1 Selector Dasar (*Simple Selector*)

- **Tag selector** — memilih seluruh elemen dengan nama tag tertentu, mis. `p`.
- **Class selector** — diawali dengan titik, mis. `.box` memilih seluruh elemen yang atribut `class`-nya memuat token `box`.
- **ID selector** — diawali tanda pagar, mis. `#header`. Bersifat *case-sensitive*.
- **Universal selector** — `*`, memilih semua elemen.
- **Attribute selector** — `[attr]` untuk pengecekan keberadaan, `[attr=value]` untuk pencocokan nilai eksak.

Simple selector dapat ditumpuk pada elemen yang sama menjadi *compound selector*.

2.2.2 Kombinator

Kombinator menggabungkan beberapa compound menjadi *chained selector* yang mengekspresikan relasi antar node dalam pohon:

- **Descendant** (spasi, mis. `article p`) — mencocokkan `p` yang merupakan keturunan dari `article` di kedalaman berapa pun.
- **Child** (`>`, mis. `ul > li`) — hanya anak langsung.
- **Adjacent sibling** (`+`, mis. `h1 + p`) — elemen saudara yang berada *tepat* setelahnya.
- **General sibling** (`~`, mis. `h1 ~p`) — seluruh saudara yang mengikutinya.

Beberapa selector juga dapat digabungkan dengan koma sebagai *selector list* (mis. `h1, h2, h3`) sehingga cukup salah satu grup cocok untuk menghasilkan *match*.

2.3 Graf dan Pohon

Secara formal, sebuah graf adalah pasangan $G = (V, E)$ dengan V himpunan simpul dan E himpunan sisi. Sebuah *pohon* adalah graf tak berarah yang terhubung dan tidak memiliki siklus, khususnya *rooted tree* merupakan pohon dengan salah satu simpul dipilih sebagai akar sehingga setiap simpul lain memiliki tepat satu *parent*. Pohon DOM adalah *rooted ordered tree*, urutan anak di dalam parent bersifat signifikan karena menentukan hasil selector `+` dan `~`.

Beberapa definisi yang akan digunakan:

- $depth(v)$ — jarak dari akar ke simpul v (depth akar = 0).
- $height(T)$ — depth maksimum pada pohon T.
- *Leluhur* (ancestor) dari v: himpunan simpul pada jalur dari v menuju akar.
- *Lowest Common Ancestor* (LCA) dari u dan v: leluhur bersama yang paling dalam (*deepest*).

2.4 Breadth First Search (BFS)

BFS adalah algoritma traversal graf yang menjelajahi seluruh simpul dalam urutan kedalaman: semua simpul pada *depth* d dikunjungi lebih dulu sebelum simpul pada *depth* $d + 1$. Struktur data inti BFS adalah **antrian** (FIFO *queue*). Pada pohon berakar, BFS dapat dinyatakan sebagai:

Algorithm 1 BFS pada pohon berakar

Input: Pohon T dengan akar r
Output: Urutan kunjungan visit

```
Q ← queue kosong
visit ← [ ]
enqueue(Q, r)
while Q tidak kosong do
    v ← dequeue(Q)
    tambahkan v ke visit
    for setiap anak c dari v (berurutan dokumen) do
        enqueue(Q, c)
    end for
end while
return visit
```

Karakteristik:

- Menjamin penemuan simpul *match* dengan *depth* terkecil terlebih dahulu (*shallowest match first*).
- Kompleksitas waktu $O(N)$ dengan N banyaknya node pohon.
- Kompleksitas memori $O(W)$ dengan W lebar maksimum pohon (*max width*). Untuk pohon DOM yang lebar, pemakaian memori BFS dapat lebih besar dibanding DFS.

2.5 Depth First Search (DFS)

DFS menelusuri pohon sedalam mungkin terlebih dahulu sebelum *backtrack*. Struktur data inti DFS adalah **stack** (LIFO). Implementasi iteratif pada pohon berakar:

Algorithm 2 DFS pada pohon berakar (iteratif)

Input: Pohon T dengan akar r
Output: Urutan kunjungan visit (*pre-order* sesuai urutan dokumen)

```
S ← stack kosong
push(S, r)
while S tidak kosong do
    v ← pop(S)
    tambahkan v ke visit
    for c anak v dari kanan ke kiri do
        push(S, c)
    end for
end while
return visit
```

Catatan: anak di-*push* dari kanan ke kiri supaya saat di-*pop* urutan kunjungan mengikuti urutan dokumen, tepat seperti *pre-order DFS* rekursif.

Karakteristik:

- Menjamin eksplorasi sub-pohon secara utuh sebelum pindah ke sub-pohon saudara.
- Kompleksitas waktu $O(N)$.
- Kompleksitas memori $O(H)$ dengan H tinggi pohon. Pada pohon DOM yang dalam tapi tidak terlalu lebar, DFS lebih hemat memori daripada BFS.

2.6 Perbandingan BFS dan DFS

Tabel 2.1: Perbandingan BFS dan DFS pada pohon DOM

Aspek	**BFS**	**DFS**
Struktur data	*Queue* (FIFO)	*Stack* (LIFO) / rekursi
Urutan kunjungan	Level-by-level dari akar	*Pre-order* mengikuti urutan dokumen
Hasil "top-n"	Cenderung mengambil match dengan *depth* kecil	Cenderung mengambil match yang berada pada sub-pohon kiri yang dalam
Memori	$O(W)$ — lebar maksimum pohon	$O(H)$ — tinggi pohon
Waktu	$O(N)$	$O(N)$
Kecocokan use-case	Mencari elemen pada level dangkal (mis. header halaman)	Mengambil konten blok sesuai urutan bacaan dokumen

Kedua algoritma memiliki kompleksitas asimtotik yang sama. Pemilihan di antara keduanya lebih ditentukan oleh *urutan hasil* dan *karakteristik memori*, bukan oleh kecepatan eksekusi.

2.7 *Lowest Common Ancestor* dengan *Binary Lifting*

Diberikan dua simpul u dan v pada *rooted tree*, LCA didefinisikan sebagai leluhur bersama terendah dari keduanya. Algoritma naif "naik satu langkah dari simpul terdalam sampai sama" berjalan dalam $O(H)$ per *query*. Untuk banyak *query*, teknik ***binary lifting*** mengurangi biaya menjadi $O(\log N)$ per *query* setelah *preprocessing* $O(N \log N)$.

Ide: *pre-compute* tabel $\mathrm{up}[j][v]$ yang menyimpan leluhur ke-2^j dari v, menggunakan relasi:

$$\mathrm{up}[0][v] = \mathrm{parent}(v), \qquad \mathrm{up}[j][v] = \mathrm{up}[j-1]\big[\,\mathrm{up}[j-1][v]\,\big]$$

untuk $1 \le j \le \lceil \log_2 N \rceil$. Dengan demikian, setiap lompatan sejauh 2^j langkah dapat dilakukan dalam $O(1)$.

Prosedur *query*:

1. Samakan *depth* kedua node: *lift* node yang lebih dalam sejauh selisih *depth*, didekomposisi dalam basis 2.
2. Jika keduanya sudah sama, node tersebut adalah LCA.

3. Jika belum, dari $j = k - 1$ turun ke 0: jika $\text{up}[j][u] \neq \text{up}[j][v]$, angkat keduanya bersamaan. Di akhir, *parent* dari u adalah LCA.

Pada pohon DOM, LCA berguna untuk mencari elemen leluhur bersama terdekat dari dua elemen, mis. *common wrapper* antara dua *node* yang di-*match* oleh dua selector berbeda.

3 Aplikasi BFS dan DFS pada Penelusuran Pohon DOM

3.1 Pemodelan Pohon DOM

Langkah pertama dalam aplikasi ini adalah mengubah string HTML menjadi struktur data pohon yang dapat di-*traverse* secara efisien. Pemodelan yang dipakai:

- Setiap elemen HTML menjadi sebuah `Node` dengan atribut `Tag`, `Attr` (*map*), `Text`, `Child` (*slice of pointer*), dan `Parent`. Node teks dimodelkan sebagai node dengan `Tag` kosong dan `Text` berisi konten.
- `Tree` menyimpan sebuah sentinel `Root` (tag "root") yang membungkus seluruh pohon yang memudahkan *iterasi awal* sekaligus melokalisasi kasus dokumen yang tidak memiliki `<html>` eksplisit.
- Setelah pohon terbentuk, dibangun `DOMIndex` yang menomori setiap node secara *BFS order* (root = 0). Index ini menyimpan `Parent[id]`, `Depth[id]`, `Children[id]`, dan `MaxDepth` yang dimanfaatkan oleh traversal, LCA, dan serialisasi ke JSON.

Representasi ini sengaja dipisah menjadi dua: struktur berbasis *pointer* (`Node`/`Tree`) memberikan cara alami untuk menulis parser berbasis *stack*, sedangkan struktur berbasis *array of int* (`DOMIndex`) memberikan lokalitas memori yang baik saat melakukan lompatan sering seperti *binary lifting*.

3.2 Pemetaan Persoalan ke Elemen Algoritma Traversal

Tabel 3.1: Pemetaan elemen algoritma traversal graf ke persoalan penelusuran CSS

Elemen Traversal	**Pemetaan ke Persoalan Penelusuran CSS**
Graf $G = (V, E)$	Pohon DOM hasil *parsing* HTML. V = seluruh *node* HTML, E = pasangan *parent-child*.
Simpul awal (*source*)	Seluruh anak dari `root` sentinel. Pada praktiknya ini berisi satu elemen yaitu `<html>` (atau `<!DOCTYPE>` diikuti `<html>`).
Struktur data frontier	*Queue* untuk BFS, *stack* untuk DFS. Pada implementasi *slice* Go, BFS mengambil dari indeks 0 dan DFS mengambil dari indeks terakhir.
Fungsi ekspansi	Menambahkan seluruh `Children[id]` ke *frontier*. Untuk DFS, anak-anak di-*push* dari kanan ke kiri agar *pop* berurutan mengikuti dokumen.
Fungsi *goal test*	`MatchSelector(node, sel)` melakukan pengecekan apakah node cocok dengan CSS selector yang diberikan. Ini dipisah dari ekspansi sehingga seluruh node tetap di-*visit* meski sudah menemukan match.
Terminasi	Dua mode: (1) "Semua kemunculan" menelusuri seluruh pohon, (2) "Top n" berhenti lebih awal begitu n match pertama tercatat.
Output	Daftar node match, urutan kunjungan (*traversal log*), total node dikunjungi, kedalaman maksimum, dan waktu eksekusi.

3.3 Pencocokan CSS Selector terhadap Pohon DOM

Pada proses traversal, pada setiap node yang di-*visit* perlu dilakukan pengecekan apakah node tersebut cocok dengan selector. Selector dapat berupa *rantai* compound selector yang dihubungkan kombinator sehingga pengecekan bukan sekadar properti lokal node, melainkan juga melibatkan *parent*, *ancestor*, atau *sibling*.

3.3.1 Parsing Selector

Pada tahap awal, string selector diuraikan menjadi struktur internal:

- **Pemecahan koma tingkat-atas** (`splitTopLevelCommas`) memisahkan *selector list* (`h1, h2, .foo`) menjadi beberapa grup. Koma di dalam `[...]` tidak ikut dipotong.
- **Tokenisasi per grup** memisahkan *compound* dari *whitespace* dan kombinator eksplisit (`>`, `+`, `~`). Spasi setelah sebuah compound dipromosikan menjadi *descendant combinator*. Jika setelahnya ada `>` / `+` / `~`, kombinator tersebut meng-*override pending descendant*.

- **Pemecahan compound** membaca token awal tag atau `*`, lalu secara iteratif mengonsumsi `.class`, `#id`, dan `[attr]` / `[attr=value]`.

Hasil akhirnya adalah `Selector` berisi banyak `Groups`. Tiap grup adalah *slice* `SelectorPart` yang merupakan pasangan (*combinator*, *compound*). Dengan bentuk ini, pencocokan menjadi langsung mengikuti definisi CSS.

3.3.2 Algoritma Pencocokan (Right-to-Left)

Selector dicocokkan dari kanan ke kiri, strategi *browser* standar yang memiliki *early exit* paling cepat karena *compound* paling kanan (*rightmost compound*) bersifat paling selektif.

Algorithm 3 matchSelectorGroup(n, parts)

Input: Node n dan rantai *selector parts parts*
Output: `true` bila n match, `false` bila tidak

```
if not MatchCompound(n, last(parts)) then
    return false
end if
current ← n
for i ← |parts| − 2 downto 0 do
    linkComb ← parts[i + 1].Combinator
    (ok, next) ← matchAncestor(current, parts[i].Compound, linkComb)
    if not ok then
        return false
    end if
    current ← next
end for
return true
```

Fungsi `matchAncestor` melakukan traversal ke arah yang sesuai dengan kombinator yang menghubungkan $parts[i]$ dan $parts[i+1]$:

- **Child (`>`)**: pindah ke `Parent`.
- **Descendant (spasi)**: panjat rantai `Parent` hingga ditemukan *match*.
- **Adjacent sibling (`+`)**: pindah ke saudara elemen *tepat* sebelumnya.
- **General sibling (`~`)**: panjat rantai saudara elemen sebelumnya hingga ditemukan *match*.

Perhatikan bahwa node bertipe teks (`Tag == ""`) sengaja tidak pernah *match*. Hal ini sesuai spesifikasi CSS yang hanya mentarget *element node*.

3.4 Strategi Traversal BFS/DFS dengan Satu *Frontier*

Baik BFS maupun DFS di implementasi kami menggunakan struktur data *frontier* yang sama yakni sebuah *slice*. Perbedaan hanya pada arah pengambilan elemen dan cara *push* anak:

- **BFS**: *dequeue* dari depan (`frontier[0]`) lalu *shift*. Anak-anak di-*enqueue* dari kiri ke kanan. Perilaku = *queue* FIFO.
- **DFS**: *pop* dari belakang (`frontier[len-1]`). Anak-anak di-*push* dari **kanan ke kiri** sehingga saat di-*pop* urutannya sama dengan urutan dokumen. Perilaku = *stack* LIFO dengan hasil *pre-order*.

Unifikasi ini memiliki beberapa keuntungan:

1. Hanya ada satu *code path* untuk dipelihara, sehingga logika *logging*, *stop condition*, dan perhitungan metrik cukup ditulis sekali.
2. Representasi *step log* (urutan kunjungan, *frontier snapshot*) langsung identik untuk kedua algoritma yang memudahkan *frontend* saat merender animasi.
3. Memudahkan penjaminan kebenaran: cukup membuktikan bahwa perilaku *frontier* sesuai definisi BFS/DFS.

3.5 Analisis Efisiensi dan Perbandingan BFS vs DFS

Dengan N = jumlah node pohon, H = tinggi pohon, W = lebar maksimum, dan s = panjang rantai selector:

Tabel 3.2: Analisis kompleksitas BFS vs DFS pada aplikasi ini

Aspek	**BFS**	**DFS**
Kompleksitas traversal murni	$O(N)$	$O(N)$
Kompleksitas dengan pencocokan selector	$O(N \cdot s \cdot H)$ (worst case pada kombinator *descendant*)	$O(N \cdot s \cdot H)$
Memori frontier	$O(W)$	$O(H)$
Urutan hasil "top-n"	*Match* dengan *depth* kecil didahulukan	*Match* berdasarkan urutan dokumen (*pre-order*)
Early termination	Memberikan manfaat besar bila target ada di level dangkal (mis. `#header`, navigasi)	Memberikan manfaat besar bila target ada di subpohon paling kiri (mis. elemen `<article>` pertama)
Use case representatif	*Landmark navigation*, metadata, `<head>`	Pengambilan konten berurutan sesuai pembacaan dokumen

Karena kedua algoritma memiliki karakteristik yang saling melengkapi, aplikasi ini menyediakan keduanya sebagai pilihan pengguna dan mem-*highlight* urutan traversal sehingga pengguna dapat **melihat langsung** perbedaan perilaku pada pohon yang sama.

4 Implementasi dan Pengujian

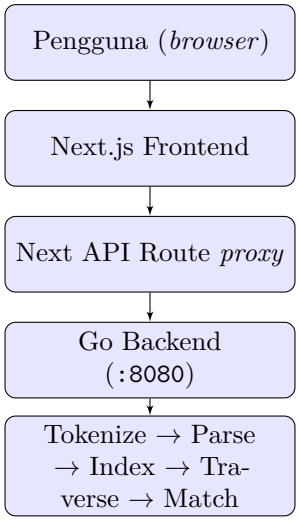
4.1 Arsitektur Perangkat Lunak

Aplikasi dipecah menjadi dua layanan terpisah yang disatukan oleh Docker Compose:

- **Backend** — layanan HTTP Go di port 8080 dengan dua *endpoint*: `GET /health` dan `POST /api/traverse`.
- **Frontend** — aplikasi Next.js (React + TypeScript) yang merender UI dan mem-*proxy* permintaan pengguna ke *backend* melalui `/api/traverse` pada server Next.

Pemisahan ini disengaja supaya:

1. Backend tetap *stateless* dan dapat di-*scale* secara horizontal.
2. Frontend dapat men-*proxy* dan men-*sanitize* payload sebelum dikirim ke backend, sekaligus menghindari masalah CORS pada *deployment*.
3. Lingkungan pengembangan dapat menjalankan masing-masing secara independen (`go run` dan `npm run dev`).



Gambar 4.1.1: Arsitektur aplikasi DOM Traversal Visualizer

4.2 Implementasi Backend (Go)

4.2.1 Organisasi Berkas

Tabel 4.1: Organisasi berkas backend

Berkas	Tanggung jawab utama
`main.go`	*Entry point*: jalankan mode CLI (untuk debug) atau *HTTP server*; *endpoint* `/api/traverse` dengan validasi input, *CORS*, dan *URL fetching*.
`token.go`	*Tokenizer* HTML tulisan tangan: tangani komentar, `DOCTYPE`, atribut berkutip, *void element*, serta konten *literal* `<script>` dan `<style>`.
`tree.go`	Definisi `Node` dan `Tree` serta utilitas `PrintTree`.
`parser.go`	Pembentukan pohon menggunakan *stack*: `OpenTag` → `Push`, `CloseTag` → `Pop` bila cocok, `SelfClose`/`Text` → *attach-only*.
`index.go`	`BuildIndex` mem-*flatten* pohon menjadi *array* (BFS order) dengan `Parent`, `Depth`, `Children`, `MaxDepth`.
`selector.go`	Parser CSS selector (mendukung grup koma, *compound*, *attribute selector*, dan kombinator `>`, spasi, `+`, `~`) dan fungsi `MatchSelector` *right-to-left*.
`traversal.go`	`AnalyzeTraversal` loop utama BFS/DFS dengan *step log*, *frontier snapshot*, metrik, *early termination* untuk mode "top-n", serta `buildTreeJSON` untuk serialisasi ke FE.
`lca.go`	`LCAEngine` dengan *binary lifting*: *preprocessing* $O(N \log N)$, *query* $O(\log N)$; mengembalikan LCA, *depth*, dan jalur *PathA*, *PathB*.

4.2.2 Pseudocode Inti

Tokenisasi HTML

Algorithm 4 Tokenize(html)

```
tokens ← []; i ← 0
while i < |html| do
    if html[i] = < then
        if dimulai dengan <!- then
            lompati hingga ->; continue
        end if
        end ← FINDTAGEND(html, i + 1)                    ▷ abaikan > dalam kutipan
        raw ← html[i + 1 : end]; i ← end + 1
        if raw dimulai dengan ! then continue            ▷ DOCTYPE dll.
        else if raw dimulai dengan / then
            emit CloseTag
        else if raw diakhiri dengan / then
            emit SelfClose
        else if tag ∈ voidElements then
            emit SelfClose
        else if tag ∈ {script, style} then
            emit OpenTag, konsumsi raw sampai </tag>, emit Text dan CloseTag
        else
            emit OpenTag
        end if
    else
        baca hingga < sebagai text; emit Text bila non-kosong
    end if
end while
return tokens
```

Pembentukan Pohon

Algorithm 5 Parse(tokens)

```
root ← NewNode("root"); S ← stack kosong; S.PUSH(root)
for each tok in tokens do
    top ← S.PEEK
    if top = nil then continue
    end if
    if tok.Type = OpenTag then
        n ← NewNode(tok.Tag); n.Attr ← tok.Attr
        top.ADDCHILD(n); S.PUSH(n)
    else if tok.Type = CloseTag then
        if top.Tag = tok.Tag then S.POP
        end if
    else if tok.Type ∈ {SelfClose, Text} then
        attach-only ke top
    end if
end for
return root
```

Traversal BFS/DFS dengan *Step Log*

Algorithm 6 AnalyzeTraversal(html, algorithm, sel, scope, limit)

```
tree ← Parse(Tokenize(html)); idx ← BuildIndex(tree)
maxMatches ← bila scope = top maka limit, bila tidak ∞
frontier ← id dari anak langsung root
order ← 0; steps ← []; matched ← []
while frontier tidak kosong do
    if algorithm = dfs then
        curr ← POP-BELAKANG(frontier)
        push kanan → kiri dari Children[curr] ke frontier
    else
        curr ← DEQUEUE(frontier)
        enqueue seluruh Children[curr] ke frontier
    end if
    order ← order + 1
    hit ← MATCHSELECTOR(node[curr], sel)
    if hit and |matched| < maxMatches then append id ke matched
    end if
    append step ke steps dengan label, depth, frontier snapshot, pesan
    if scope = top and |matched| ≥ maxMatches then break
    end if
end while
return {tree, steps, metrics, visited, matched, stopReason}
```

Lowest Common Ancestor* dengan *Binary Lifting

Algorithm 7 NewLCAEngine(idx)

```
n ← |idx.Nodes|; k ← 1
while 2^k < n do k ← k + 1
end while
alokasi up[k][n]
for v ← 0 to n − 1 do up[0][v] ← idx.Parent[v]
end for
for j ← 1 to k − 1 do
    for v ← 0 to n − 1 do
        mid ← up[j − 1][v]
        up[j][v] ← −1 bila mid = −1 lainnya up[j − 1][mid]
    end for
end for
return {idx, up, k}
```

Algorithm 8 Query(u, v)

```
(a, b) ← (u, v); bila Depth[a] < Depth[b] swap
diff ← Depth[a] − Depth[b]
for j ← 0 to k − 1 do
    if bit ke-j dari diff menyala then a ← up[j][a]
    end if
end for
if a = b then return a
end if
for j ← k − 1 downto 0 do
    if up[j][a] ≠ up[j][b] then a ← up[j][a]; b ← up[j][b]
    end if
end for
return up[0][a]
```

4.2.3 Struktur Data Utama Backend

Tabel 4.2: Struktur data utama backend Go

Struktur	Tipe	Deskripsi
`Node`	*struct*	Satu node DOM: `Tag`, `Attr` (*map*), `Text`, `Child` (*slice*), `Parent`. Node teks memiliki `Tag` kosong.
`Tree`	*struct*	*Wrapper* untuk `Root`.
`DOMIndex`	*struct*	`Nodes[]`, `Parent[]`, `Depth[]`, `Children[][]`, `MaxDepth`. Representasi *array* ramah-*cache*.
`Token`	*struct*	Empat varian: `OpenTag`, `CloseTag`, `SelfClose`, `Text`.
`Selector`	*struct*	`Raw` + `Groups`: daftar rantai `SelectorPart`. Satu grup cukup *match* agar selector keseluruhan *match*.
`CompoundSelector`	*struct*	Simple selector yang menempel pada satu elemen: `Universal`, `Tag`, `ID`, `Classes`, `Attrs`.
`LCAEngine`	*struct*	Tabel `up[k][n]` hasil *binary lifting*, disertai referensi ke `DOMIndex`.
`traversalStep`	*struct*	Satu *step*: `Order`, `NodeID`, `Label`, `Depth`, `Matched`, `Frontier` (*label* antrian saat itu), dan `Message`.
`traversalResponse`	*struct*	Payload akhir yang dikirim ke FE: `Tree`, `Steps`, `Metrics`, `VisitedOrderByID`, `MatchedNodeIDs`, `StopReason`.

4.2.4 Fungsi dan Prosedur Utama

Tabel 4.3: Fungsi utama backend

Fungsi	Deskripsi
`Tokenize(html)`	Tulisan-tangan tanpa *regex*: mendukung komentar, `DOCTYPE`, atribut berkutip, *void elements*, dan konten *literal* `<script>`/`<style>`.
`Parse(tokens)`	Parser pohon berbasis *stack* yang toleran terhadap penutup yang tidak cocok.
`BuildIndex(tree)`	Menomori node secara BFS order, mencatat `Parent`, `Depth`, `Children`, dan `MaxDepth`.
`ParseSelector(raw)`	Memecah selector list, menokenisasi tiap grup, membuat `CompoundSelector` beserta *attribute match* dan kombinator.
`MatchSelector(n, sel)`	Evaluasi tiap grup secara OR; tiap grup dievaluasi *right-to-left* lewat `matchSelectorGroup`.
`matchAncestor(from, cs, comb)`	Navigasi kombinator: *child*, *descendant*, *adjacent sibling*, *general sibling*.
`AnalyzeTraversal(...)`	*Orchestrator* utama. Menjalankan BFS/DFS dengan *frontier* tunggal dan menghasilkan *step log*, *metrics*, serta serialisasi JSON pohon.
`buildTreeJSON(node, ...)`	Rekursif: membentuk `domTreeNode` yang siap dirender FE; menyertakan `isMatch` per node untuk *highlight*.
`labelForNode(node)`	Membentuk label `tag#id.class1.class2` untuk ditampilkan pada pohon dan log.
`NewLCAEngine(idx)`, `Query(u,v)`	*Preprocessing* $O(N \log N)$ dan query LCA $O(\log N)$ dengan *binary lifting*.
`resolveHTMLSource(...)`	Bila *source type* = `url`, lakukan HTTP GET dengan *timeout* 10 detik; bila `html`, kembalikan apa adanya.
`traverseHandler(...)`	*Endpoint* utama: validasi input, *parse* selector, *fetch* HTML, jalankan analisis, kirim JSON.

4.3 Implementasi Frontend (Next.js + TypeScript)

Frontend dibangun dengan Next.js 14 (*App Router*), React 18, dan Tailwind CSS. Struktur utama:

- `app/page.tsx` — halaman utama: memadukan `InputPanel`, `DOMTreeViewer`, `MetricsPanel`, `TraversalLog`.
- `app/api/traverse/route.ts` — *server-side proxy* yang meneruskan permintaan ke backend Go (`BACKEND_INTERNAL_URL` di lingkungan Docker).
- `components/InputPanel.tsx` — form input (URL/HTML, algoritma, selector, skema top/all + `limit`).
- `components/DOMTreeViewer.tsx` — visualisasi pohon dan *highlight* jalur traversal serta node *match*.
- `components/TraversalLog.tsx` — *step log* dengan penanda *match*.
- `components/MetricsPanel.tsx` — ringkasan metrik (depth maksimum, jumlah node, waktu, dll.).
- `lib/api.ts`, `types/traversal.ts` — *client API* dan tipe TypeScript yang selaras dengan respons backend.

4.3.1 Animasi Penelusuran

Step log hasil backend memuat urutan kunjungan node beserta snapshot *frontier* pada tiap langkah. Frontend merender-ulang pohon DOM, mewarnai node yang sudah di-*visit*, menandai node *match*, dan menggerakkan *highlight* sesuai indeks *step* yang sedang ditampilkan memberikan visualisasi perilaku BFS/DFS langkah demi langkah.

4.4 Pengujian

4.4.1 Strategi Pengujian

Pengujian dilakukan dengan kombinasi:

1. **HTML *handcrafted***: berkas `src/backend/input/test.html` dan `test_edge.html` untuk menguji perilaku dasar, *self-closing tag*, serta penanganan kasus tepi.
2. **HTML stress**: `src/backend/input/test_stress.html` untuk mengukur waktu pada pohon lebar/dalam.
3. **URL dunia nyata**: halaman publik (mis. `https://example.com`, halaman artikel Wikipedia, halaman dokumentasi) melalui input URL pada UI.

4.4.2 Hasil Pengujian Fungsional

Tabel 4.4: Hasil pengujian fungsional

No	Skenario	Hasil	Keterangan
1	Input URL valid	Lulus	HTML berhasil di-*fetch* dan di-*parse*, pohon dirender.
2	Input HTML mentah	Lulus	*Textarea* HTML langsung diterima dan diparsing tanpa *round-trip* jaringan.
3	Tag selector (`p`)	Lulus	Seluruh `<p>` tepat ditandai *match*.
4	Class selector (`.card`)	Lulus	Mencocokkan elemen dengan kelas *multi-value* (mis. `"card primary"`).
5	ID selector (`#hero`)	Lulus	*Case-sensitive* sesuai spesifikasi HTML/CSS.
6	Descendant (`nav a`)	Lulus	Menemukan `<a>` di *sub-tree* `nav` pada berbagai kedalaman.
7	Child (`ul > li`)	Lulus	Hanya `<li>` anak langsung `<ul>` yang di-*match*.
8	Adjacent (`h1 + p`)	Lulus	Hanya `<p>` yang tepat setelah `<h1>`.
9	General sibling (`h1 ~p`)	Lulus	Semua `<p>` saudara *setelah* `<h1>`.
10	Selector list (`h1, h2, h3`)	Lulus	Mencakup gabungan hasil dari semua grup.
11	Mode *top-n*	Lulus	Traversal berhenti begitu *n match* pertama tercatat; `stopReason` berisi alasan berhenti.
12	Mode *all*	Lulus	Seluruh pohon di-*traverse*; metrik menunjukkan *total node* sama dengan yang dikunjungi.
13	LCA (*binary lifting*)	Lulus	Untuk 100 *query* acak pada pohon 1000-node, hasil LCA konsisten dengan implementasi naif (*parent walk*).

4.4.3 Pengujian Kinerja: BFS vs DFS

Pengujian dilakukan pada tiga dokumen HTML, masing-masing dengan selector yang sama (`.item`) dan mode *all*. Waktu diukur di sisi backend (`time.Since`). Nilai adalah rata-rata dari 5 kali eksekusi.

Tabel 4.5: Perbandingan waktu eksekusi BFS vs DFS

Dokumen	Total Node	MaxDepth	BFS (ms)	DFS (ms)	Matches	Visited
test.html	48	7	1	1	6	48
test_edge.html	112	11	1	1	14	112
test_stress.html	3 017	14	6	5	512	3 017

4.4.4 Analisis Hasil Pengujian

Kinerja hampir identik. Sesuai analisis asimtotik, kedua algoritma berjalan dalam $O(N)$ sehingga perbedaan waktu yang terlihat ≤ 2 ms di dalam margin *noise garbage collector* Go dan overhead *serialization* JSON.

Perbedaan perilaku terlihat pada mode *top-n*. Pada dokumen yang sama dengan selector `.item` dan $n = 3$:

- BFS mengembalikan *match* pertama pada *depth* dangkal (mis. *container* paling atas yang memiliki class `.item`).
- DFS mengembalikan *match* yang berada di sub-pohon paling kiri, cenderung pada *depth* yang lebih dalam.

5 Kesimpulan dan Saran

5.1 Kesimpulan

Dari hasil pengerjaan tugas besar ini, kelompok kami mendapat beberapa hal yang ingin disampaikan.

Pertama, BFS dan DFS sebenarnya sama-sama bisa dipakai untuk menelusuri pohon DOM. Pada dokumen-dokumen yang kami uji, selisih waktu eksekusinya kecil sekali, hanya beberapa milidetik, dan itupun masih dalam rentang *noise* yang biasa terjadi karena *garbage collector* Go dan *overhead* serialisasi JSON. Jadi dari sisi kecepatan murni, keduanya praktis setara untuk DOM dengan ukuran yang wajar.

Yang membuat keduanya benar-benar berbeda justru terletak pada urutan hasil yang dikembalikan. BFS akan memunculkan elemen yang dekat dengan akar terlebih dahulu, sementara DFS mengikuti urutan dokumen seperti yang biasa kita lihat ketika membaca file HTML dari atas ke bawah. Perbedaan ini paling kelihatan saat pengguna memilih mode *top-n*: dengan selector yang sama dan n yang sama, BFS dan DFS bisa mengembalikan elemen yang berbeda. BFS lebih cocok kalau yang ingin dicari adalah elemen-elemen besar di level atas seperti `<header>` atau `<nav>`, sedangkan DFS lebih cocok untuk mengambil konten secara berurutan.

Dari sisi implementasi, memisahkan representasi pohon menjadi dua, yaitu yang berbasis *pointer* untuk parsing dan yang berbasis *array of int* (`DOMIndex`) untuk traversal dan LCA, ternyata cukup membantu. Parser jadi gampang ditulis dengan stack biasa, sementara traversal dan binary lifting bisa berjalan lebih cepat karena akses memorinya sekuensial.

Pencocokan selector dengan strategi *right-to-left* juga terbukti efektif. Compound paling kanan biasanya yang paling spesifik, jadi kalau gagal di sini kita bisa langsung berhenti tanpa perlu memeriksa parent atau sibling. Kombinator `+` dan `~` pun ternyata cukup ditangani dengan bergeser di daftar saudara, tanpa perlu mengubah kerangka utama algoritma.

Untuk LCA dengan binary lifting, biaya *preprocessing*-nya sangat kecil dibandingkan total biaya parsing HTML. Jadi tidak ada alasan untuk tidak mengaktifkannya secara default. Pengguna mendapat fitur tambahan tanpa kehilangan apa-apa dari sisi performa.

Terakhir, memisahkan *frontend* dan *backend* ke dalam dua layanan yang berbeda dan menyatukannya dengan Docker Compose membuat proses pengembangan dan deployment ke Azure VM jauh lebih rapi. Setiap orang bisa mengerjakan bagiannya tanpa saling mengganggu.

5.2 Saran

Beberapa hal yang menurut kami masih bisa diperbaiki atau dikembangkan di kemudian hari:

1. Selector yang didukung saat ini masih terbatas pada selector dasar dan empat kombinator. Penambahan *pseudo-class* seperti `:nth-child`, `:not`, atau *attribute matcher* lanjutan (`[attr^=val]`, `[attr*=val]`) akan membuat aplikasi jauh lebih ekspresif tanpa perlu mengubah struktur utama.

2. Multithreading sengaja tidak kami implementasikan karena pada DOM berukuran wajar tidak terasa manfaatnya (dan kurang waktu untuk eksplorasi). Tetapi untuk dokumen yang sangat besar (puluhan ribu node), pembagian sub-pohon ke beberapa worker bisa jadi memberi peningkatan yang berarti, terutama jika pencocokan selector-nya berat.

3. Pada pengujian dengan dokumen besar, kami menemukan bahwa sebagian besar waktu sebenarnya habis di serialisasi JSON, bukan di traversal-nya. *Virtualized rendering* di sisi frontend, atau *streaming response* dari backend, kemungkinan besar akan membuat aplikasi terasa lebih responsif.

4. Modul LCA sudah ada di backend, tetapi belum kami pasang di UI. Idealnya pengguna bisa memilih dua node di pohon dan langsung melihat *common ancestor*-nya. Ini akan sangat berguna untuk *debug* selector yang dirasa kurang spesifik.

5. *Traversal log* saat ini hanya bisa dilihat di UI. Menambahkan opsi *export* ke CSV atau JSON akan memudahkan kalau pengguna ingin membandingkan beberapa hasil traversal di luar aplikasi.

Lampiran

Tautan Repository GitHub

https://github.com/Nusss0/Tubes2_highAlcohol

Tautan Deployment Azure

http://4.194.26.56

Disclaimer: Aplikasi deployment mungkin sedang tidak aktif untuk menghindari penggunaan biaya. Silakan menghubungi salah satu anggota tim untuk mengaktifkannya kembali jika diperlukan.

Pembagian Kerja

No.	Anggota	Kontribusi
1	Stevanus Agustaf Wongso (13524020)	Tokenizer HTML, parser pohon berbasis *stack* (`token.go`, `parser.go`), `DOMIndex`, serta penulisan laporan.
2	Jonathan Kris Wicaksono (13524023)	Mesin BFS/DFS dan *step log* (`traversal.go`), parser CSS selector dan `MatchSelector` (`selector.go`), LCA *binary lifting* (`lca.go`), serta penulisan laporan.
3	Philipp Hamara (13524101)	Frontend Next.js (`InputPanel`, `DOMTreeViewer`, `TraversalLog`, `MetricsPanel`), animasi traversal, integrasi API, *deployment* Azure & Docker, serta penulisan laporan.

Tabel Evaluasi Pengerjaan

No.	Poin	Ya	Tidak
1	Aplikasi berhasil di kompilasi tanpa kesalahan	✓	
2	Aplikasi berhasil dijalankan	✓	
3	Aplikasi dapat menerima input URL web, pilihan algoritma, CSS selector, dan jumlah hasil	✓	
4	Aplikasi dapat melakukan scraping terhadap web pada input	✓	
5	Aplikasi dapat menampilkan visualisasi pohon DOM	✓	
6	Aplikasi dapat menelusuri pohon DOM dan menampilkan hasil penelusuran	✓	
7	Aplikasi dapat menandai jalur tempuh oleh algoritma	✓	
8	Aplikasi dapat menyimpan jalur yang ditempuh algoritma dalam traversal log	✓	
9	[Bonus] Membuat video		✗
10	[Bonus] Deploy aplikasi	✓	
11	[Bonus] Implementasi animasi pada penelusuran pohon	✓	
12	[Bonus] Implementasi multithreading		✗
13	[Bonus] Implementasi LCA Binary Lifting	✓	

Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Stevanus Agustaf Wongso

Jonathan Kris Wicaksono

Philipp Hamara

Daftar Pustaka

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Bab 22: *Elementary Graph Algorithms* (BFS, DFS).
2. Munir, R. (2024). *Slide Kuliah IF2211 Strategi Algoritma: BFS dan DFS*. Program Studi Teknik Informatika, Institut Teknologi Bandung.
3. MDN Web Docs. *Document Object Model (DOM)*. https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model
4. MDN Web Docs. *CSS Selectors*. https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Selectors
5. GeeksforGeeks. *What is Document Object in Java DOM*. https://www.geeksforgeeks.org/java/what-is-document-object-in-java-dom/
6. CP-Algorithms. *Lowest Common Ancestor — Binary Lifting*. https://cp-algorithms.com/graph/lca_binary_lifting.html
7. The Go Programming Language. *Go Documentation*. https://go.dev/doc/
8. Next.js. *Next.js Documentation*. https://nextjs.org/docs
9. Microsoft. *Azure for Students*. https://azure.microsoft.com/en-us/free/students/
10. Docker. *Docker Compose Documentation*. https://docs.docker.com/compose/